

REPORT CS 343 LAB 3

Group 38

- Raj Nandani Kumari: 230123048
- Neha Sellappan Devadas: 230123040
- Subhashree Kedia: 230123063
- Aditya Samal: 230123002

Our code files:  CS343_Lab_3

Task 1:

Task Overview

We have to add fundamental Inter-Process Communication (IPC) mechanisms to the xv6 kernel:

- Semaphores for managing concurrent access to resources
- Shared memory for efficient data sharing between processes

Methodology

MAIL BOX

❖ Global Mailbox Table

A global table maintains all mailboxes in the system. Each mailbox contains:

- ❖ A **buffer** (`buf[ ]`) to store messages.
- ❖ Pointers (`head`, `tail`) to track positions in the circular queue.
- ❖ A **count** of how many messages are currently stored.
- ❖ Sleep/wakeup channels (`not_empty`, `not_full`) to block processes when the mailbox is empty or full.
- ❖ A lock to synchronize concurrent access.

❖ Creation (`mbox_create`)

- ❖ When a process creates a mailbox with a given key, the kernel either returns the existing mailbox ID (if already created) or allocates a new one.
- ❖ Each mailbox is initialized with an empty buffer and ready for use.

❖ Sending Messages (`mbox_send_k`)

- ❖ A process trying to send a message acquires the mailbox lock.
- ❖ If the mailbox buffer is full, the sender is put to sleep on the `not_full` channel until space becomes available.
- ❖ Once space is free, the message is added at the `tail` of the buffer, and the `tail` pointer is advanced in circular fashion.
- ❖ The message count is incremented, and any receivers waiting on `not_empty` are woken up.
- ❖ The lock is released.

❖ Receiving Messages (`mbox_recv_k`)

- ❖ A process trying to receive, acquires the mailbox lock.
- ❖ If the mailbox buffer is empty, the receiver sleeps on the `not_empty` channel until a message arrives.

- ❖ Once a message is available, it is read from the **head** of the buffer, and the **head** pointer is advanced circularly.
- ❖ The message count is decremented, and any senders waiting on **not_full** are woken up.
- ❖ The lock is released, and the message is returned to the caller.
- ❖ **Synchronization and Safety**
 - ❖ Locks ensure that multiple processes can safely send/receive concurrently without race conditions.
 - ❖ Sleep/wakeup channels provide efficient blocking, avoiding busy waiting.
 - ❖ The circular buffer ensures efficient use of space inside the fixed-size capacity (**MBOX_CAP**).

SHARED MEMORY

- ❖ **Global Registry**

A global table stores all active shared memory regions. Each entry contains the key, a pointer to the physical page, and a reference counter indicating how many processes are currently attached.
- ❖ **Creation (**shm_create**)**
 - ❖ When the first process requests a new key, the kernel allocates a physical page, zeros it, and records it in the registry.
 - ❖ If another process later requests the same key, the registry entry is reused; no new memory is allocated.
- ❖ **Attachment (**shm_get**)**
 - ❖ A process attaching to a shared region is given a new virtual address within its own address space.
 - ❖ The kernel maps the region's physical page to this virtual address.
 - ❖ The reference counter in the registry is incremented.
 - ❖ As a result, different processes may have different virtual addresses, but all point to the **same physical page**.
- ❖ **Detachment (**shm_close**)**
 - ❖ When a process closes a region, the mapping is removed from its page table.
 - ❖ The registry reference counter is decremented.
 - ❖ If this was the last process using the region (reference count reaches zero), the kernel frees the physical page and removes the registry entry.
- ❖ **Sharing Semantics**

Because all attached processes map to the same physical page, any write performed by one process is immediately visible to others. This provides efficient inter-process communication without copying data between address spaces

Code Overview

The following changes were made in **kernel**:

Created a file **shm.c** (added to [shm.c File](#)) which contains and the function we added to **def.h** as well

- **struct shm_region**
- **static struct shmtable**
- Function **shminit** (added to **main.c**)
- Function **shm_create**

- Function `shm_get`
 - `shm_get` attaches an existing shared memory region to the calling process, making it accessible at a new virtual address, and handles all necessary bookkeeping and error cases.
- Function `shm_close`
 - It detaches a shared memory region from a process, and frees the physical memory only when no process is using it anymore.

Created a file `mbox.c` ([mbox.c File](#)) which contains and the function we added to `def.h` as well

- `struct mailbox`
- `static struct mtable`
- Function `mboxinit` (added to `main.c`)
- Function `mbox_create`
- Function `mbox_send_k`
 - This function safely adds a message to the mailbox, blocking if the mailbox is full, and wakes up receivers if needed.
- Function `mbox_recv_k`
 - This function safely retrieves a message from the mailbox, blocking if the mailbox is empty, and wakes up senders if needed.
-

In the file `sysproc.c` in kernel the following changes were made and corresponding changes required for these were made in `syscall.h` (where system call number were assigned) and `syscall.c` (where we mapped syscall numbers from `syscall.h` to the function that handles the system call) :

```
uint64
sys_shm_create(void)
{
    int key;
    argint(0, &key);
    return shm_create(key);
}
```

```
uint64
sys_shm_get(void){
    int key;
    argint(0, &key);
    return shm_get(key);
    // returns VA or 0
}
```

```
uint64
sys_shm_close(void){
    int key;
    argint(0, &key);
    return shm_close(key);
}
```

```
uint64
sys_mbox_create(void){
    int key;
    argint(0, &key);
    return mbox_create(key);
}
```

```
uint64
sys_mbox_send(void){
    int id, val;
    argint(0, &id);
    argint(1, &val);
    return mbox_send_k(id, val);
}
```

```
uint64
sys_mbox_recv(void){
    int id;
    uint64 up; // user pointer to int
    int val;
    argint(0, &id);
    argaddr(1, &up);
    if (mbox_recv_k(id, &val) < 0) return -1;
    if (copyout(myproc()->pagetable, up, (char *)&val, sizeof(int)) < 0) return -1;
    return 0;
}
```

In the file `proc.h` in kernel the following changes were made

```

struct shm_att {
    int    used;
    int    key;
    uint64 va;
    // where this shared page
    // is mapped in this proc
};

```

```

#define MAX_SHM_PER_PROC 16
struct shm_att shm[MAX_SHM_PER_PROC];

```

We added `struct shm_att` so each process can keep track of which shared memory regions it has attached, their keys, and where they are mapped in its address space.

The changes in the right image were added so each process can track and manage its own shared memory regions.

We added `shminit()` and `mboxinit()` in `main.c` to initialize the shared memory and mailbox subsystems at boot.

The following changes were made in `user`:

We created two test files `mbtest.c` and `shmtest.c` for verifying the mailbox and shared memory implementation respectively.

[shmtest.c](#): ([shmtest.c File](#))

This program shows how two related processes can talk to each other through shared memory. Think of it like creating a shared notebook that both parent and child can read from and write to.

- Creates a shared memory space using a unique key (like a room number)
- Sets up four memory slots: one for data and three for coordination signals
- Uses special compiler instructions to ensure memory changes are actually visible

The parent and child take turns writing and reading, like passing notes back and forth. They use simple flags to let each other know when it's safe to proceed with the next step.

- Parent writes the first number (1234) and raises a "ready" flag
- Child waits for the signal, reads the number, then writes its own (5678)
- Both use small delays to avoid constantly checking flags and wasting computer resources.

Output:

```

Parent: wrote 1234 into shared page
Parent: signaled child, waiting for child to finish
Child: starting
Child: waiting for parent signal
Child: parent signaled, reading value
Child: read 1234 from shared page
Child: wrote 5678 into shared page
Child: exiting
Parent: final value = 5678 in shared page

```

[mbtest.c](#): ([mbtest.c File](#))

This program is a test of message passing between a parent and child process using a mailbox.

1. At the start, it creates a mailbox.
2. The program then creates a child process using a fork.. Now there are two processes:
 - the parent
 - the child
 Both have access to the same mailbox.
3. A total of six messages are exchanged. The exchange follows a simple protocol:
 - In the first half of the sequence, the parent sends three messages and the child receives them.
 - In the second half, the child sends three messages and the parent receives them.
4. To make sure the messages don't all rush at once, there are artificial delays, essentially short pauses so the output looks orderly.
5. The parent sends simple numbers (10, 20, 30), and the child sends back different numbers (like 400, 500, 600). Each side prints what it sends or receives, so you can follow the conversation.
6. Once all six messages are exchanged, both processes announce they are finished, and the test ends.

Output:

Mailbox Test (Bidirectional)	Child: Sending message 3: 400
Created mailbox with ID: 0	Parent: Received message 3: 400
Parent: Starting messages	Child: Sending message 4: 500
Parent: Sending message 0: 10	Parent: Received message 4: 500
Child: Received message 0: 10	Child: Sending message 5: 600
Parent: Sending message 1: 20	Parent: Received message 5: 600
Child: Received message 1: 20	Child: Finished messages
Parent: Sending message 2: 30	Parent: Finished messages
Child: Received message 2: 30	Mailbox test completed

Task 2.

The objective of this task was to combine **shared memory** and **message-passing mailboxes** to solve a synchronization and data-sharing problem. Two processes (Process A and Process B) must traverse an *intertwined path* stored in shared memory. Neither process knows its own path in advance; instead, each must rely on the other process for directions, communicated through mailboxes. Each of them has to wait for the other to respond with the next location information.

Code files: [master.c](#) , [Process.c](#)

Methodology

Shared Memory Maze Structure

- A shared memory page holds a `maze_t` structure(size 4096B), which includes:
 - **Starting positions** for Process A and Process B.
 - A **lookup table (`data[]`)** where each entry defines the intertwined path.
- The intertwined nature means:
 - When Process A is at position `x`, the value stored in `data[x]` is the next position for Process B.

- Conversely, when Process B is at position `y`, `data[y]` defines the next position for Process A.
- The traversal ends when both processes encounter a special **end marker** (`0xFFFF`).

Master Program (`master.c`)

- Creates the shared memory segment (`shm_create`) and initializes the maze with intertwined paths.
- Sets starting positions for Process A (0) and Process B (5).
- Initializes two mailboxes:
 - **A → B mailbox** for messages from Process A to Process B.
 - **B → A mailbox** for messages from Process B to Process A.
- Forks two child processes (`exec process`), one for each role, passing:
 - Role (**A** or **B**).
 - Starting position.
 - Keys for send and receive mailboxes.
- Waits for both processes to finish and then cleans up shared memory.

Process Program (`process.c`)

Each process follows a strict communication protocol to ensure deadlock-free traversal:

- 1. Communication Protocol**
 - **Process A** always **sends first, then receives**.
 - **Process B** always **receives first, then sends**.
 - This fixed order prevents deadlocks (avoiding the case where both wait to receive indefinitely).
- 2. Traversal Steps**
 - Look up the current position in shared memory to compute the other process's next position.
 - Send this value to the other process via the send mailbox.
 - Receive its own next position from the peer through the receive mailbox.
 - Update its local position accordingly.
 - Mark completion if the end marker is reached.
- 3. Termination**
 - The loop continues until both processes reach the end marker.
 - Once completed, the process detaches from shared memory (`shm_close`) and exits.

Synchronization and Safety

- **Mailboxes** ensure strict ordering of send/receive messages.
- **Shared memory** guarantees both processes access the same maze structure without copying.
- **Deadlock prevention** is achieved by enforcing asymmetric communication roles (send-first vs. receive-first).

Code overview

Master.c

```

void initialize_maze(maze_t* maze) {
    for (int i = 0; i < 1022; i++) {
        maze->data[i] = END_MARKER;
    }

    maze->process_a_start = 0;
    maze->process_b_start = 5;

    // maze->data is initialised as:
    // maze->data[current position of 1 player] = next position of other player
    maze->data[0] = 1;
    maze->data[2] = 3;
    maze->data[4] = 7;
    maze->data[6] = END_MARKER;

    maze->data[5] = 2;
    maze->data[1] = 4;
    maze->data[3] = 6;
    maze->data[7] = END_MARKER;

    printf("Master: Initialized intertwined maze paths:\n");
    printf(" Process A will traverse: 0 -> 2 -> 4 -> 6 -> END (guided by B)\n");
    printf(" Process B will traverse: 5 -> 1 -> 3 -> 7 -> END (guided by A)\n");

    printf("Master: Lookup table:\n");
    for (int i = 0; i <= 7; i++) {
        if (maze->data[i] == END_MARKER)
            printf(" data[%d] = END_MARKER\n", i);
        else
            printf(" data[%d] = %d\n", i, maze->data[i]);
    }
}

```

- The func assigns starting positions for Process A and Process B and fills the shared memory table with an intertwined path structure.
- Each entry in the `data[ ]` array maps one process's current position to the other process's next position, ensuring coordinated traversal.

```

// Create mailboxes for inter-process communication
int mbox_a_to_b = mbox_create(MBOX_A_TO_B_KEY);
int mbox_b_to_a = mbox_create(MBOX_B_TO_A_KEY);

```

- Two mailboxes are created to enable bidirectional communication between the processes.
- One mailbox handles messages from Process A to Process B, while the other handles messages from Process B to Process A.

```

// Fork process A
int pid_a = fork();
if (pid_a == 0) {
    // Child process - this will be Process A
    char start_pos[10];
    char send_key[10];
    char rcv_key[10];
    itoa(maze->process_a_start, start_pos, 10);
    itoa(MBOX_A_TO_B_KEY, send_key, 10);
    itoa(MBOX_B_TO_A_KEY, rcv_key, 10);
    char* args[6];
    args[0] = "process";
    args[1] = "A";
    args[2] = start_pos;
    args[3] = send_key;
    args[4] = rcv_key;
    args[5] = 0;
    exec("process", args); // execute process.c(f
    printf("Master: Failed to exec process A\n");
    exit(1);
}

```

- A child process is created using `fork()`, which is designated as **Process A**.
- The child process is then executed with `exec("process", args)`, passing its role, starting position, and mailbox keys as arguments.
- Similarly done for **Process B**.

process.c

- This code is for individual execution of a process traversing and communicating via mailboxes and the intertwined path is stored on shared memory.
- 2 mailboxes are used per process, one for sending messages and one for receiving.

For Process A: (the first message sender)

- Sending peer information

```
if (role_send_first) {
    // --- SEND: Look up MY CURRENT position to get OTHER's next position ---
    int next_for_other;
    if (my_done || my_pos < 0 || my_pos >= DATA_SIZE) {
        next_for_other = END_MARKER;
    } else {
        next_for_other = shared_data[my_pos];
    }

    printf("[proc %d role %c] SEND -> looking up my current pos %d -> telling peer to go to %s (%d)\n",
        pid_cur, role_char, my_pos,
        next_for_other == END_MARKER ? "END_MARKER" : "position",
        next_for_other);

    if (mbox_send(send_mbox, next_for_other) < 0) {
        printf("[proc %d] mbox_send failed\n", pid_cur);
        exit(1);
    }

    if (next_for_other == END_MARKER) {
        other_done = 1;
    }
}
```

It will first look up its own position and then send peer's next location information to peer via mailbox.

Also checking here if peer has reached the end location.

- Receiving information from peer

```
// --- RECV: Get my next position from other process ---
int my_next_pos;
if (mbox_rcv(recv_mbox, &my_next_pos) < 0) {
    printf("[proc %d] mbox_rcv failed\n", pid_cur);
    exit(1);
}

printf("[proc %d role %c] RECV <- peer tells me to go to %s (%d)\n",
    pid_cur, role_char,
    my_next_pos == END_MARKER ? "END_MARKER" : "position",
    my_next_pos);

// NOW update my position after sending instruction based on old position
if (my_next_pos == END_MARKER) {
    my_done = 1;
    printf("[proc %d role %c] I have reached the end!\n", pid_cur, role_char);
} else {
    printf("[proc %d role %c] moving from %d to %d\n", pid_cur, role_char, my_pos, my_next_pos);
    my_pos = my_next_pos;
}
```

It will wait for the other process's message to go to the next location, and move to the next location.

Also checking here if A has reached the end location.

For Process B: (the second message sender)

- Receiving peer information

```
// --- RECV first: Get my next position from other process ---
int my_next_pos;
if (mbox_recv(recv_mbox, &my_next_pos) < 0) {
    printf("[proc %d] mbox_recv failed\n", pid_cur);
    exit(1);
}

printf("[proc %d role %c] RECV <- peer tells me to go to %s (%d)\n",
        pid_cur, role_char,
        my_next_pos == END_MARKER ? "END_MARKER" : "position",
        my_next_pos);
```

It will wait for the other process's message to go to the next location and does not immediately update the location.

- Sending peer information then update location

```
// --- SEND: Look up MY CURRENT position (before updating) to get OTHER's next position ---
int next_for_other;
if (my_done || my_pos < 0 || my_pos >= DATA_SIZE) {
    next_for_other = END_MARKER;
} else {
    next_for_other = shared_data[my_pos];
}

printf("[proc %d role %c] SEND -> looking up my current pos %d -> telling peer to go to %s (%d)\n",
        pid_cur, role_char, my_pos,
        next_for_other == END_MARKER ? "END_MARKER" : "position",
        next_for_other);

if (mbox_send(send_mbox, next_for_other) < 0) {
    printf("[proc %d] mbox_send failed\n", pid_cur);
    exit(1);
}

if (next_for_other == END_MARKER) {
    other_done = 1;
}

// NOW update my position after sending instruction based on old position
if (my_next_pos == END_MARKER) {
    my_done = 1;
    printf("[proc %d role %c] I have reached the end!\n", pid_cur, role_char);
} else {
    printf("[proc %d role %c] moving from %d to %d\n", pid_cur, role_char, my_pos, my_next_pos);
    my_pos = my_next_pos;
}
```

It will send the message for the next location to the peer before updating, then update its position based on the old position.

Also, checking if it has reached the end.

Exiting traversal loop

```
if (my_done && other_done) {
    printf("[proc %d role %c] both processes done -> exiting\n", pid_cur, role_char);
    break;
}
```

Exit the loop if both have reached the end.

Output

```

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ master
Master: Starting maze traversal game setup...
Master: Created shared memory (ID: 0)
Master: Initialized intertwined maze paths:
  Process A will traverse: 0 -> 2 -> 4 -> 6 -> END (guided by B)
  Process B will traverse: 5 -> 1 -> 3 -> 7 -> END (guided by A)
Master: Lookup table:
  data[0] = 1
  data[1] = 4
  data[2] = 3
  data[3] = 6
  data[4] = 7
  data[5] = 2
  data[6] = END_MARKER
  data[7] = END_MARKER
Master: Created mailboxes - A->B (ID: 0), B->A (ID: 1)
Master: Launched Process A (PID: 4) and Process B (PID: 5)
Master: Waiting for both processes to complete [mazper oc [p4trroc a 5 rrole Bol]v e Ae] mrsamyl_.s.yta_s.rtart
=t=0 send_m5b oxs=0e nrecv_d_mmbbox=1 recvx=_1
mb[oprocx=0
  4 role A] SEND -> looking up my current pos 0 -> telling peer to go to position (1)
[proc 5 role B] RECV <- peer tells me to go to position (1)
[proc 5 role B] SEND -> looking up my current pos 5 -> telling peer to go to position (2)
[p[roc pr5o c 4r olreol eB] Amov] Ring frEoCvM 5<- p teo 1
er tells me to go to position (2)
[proc 4 role A] moving from 0 to 2
[proc 4 role A] SEND -> looking up my current pos 2 -> telling peer to go to position (3)
[proc 5 role B] RECV <- peer tells me to go to position (3)
[proc 5 role B] SEND -> looking up my current pos 1 -> telling peer to go to position (4)
[proc 5 role B] moving from 1 to 3
[proc 4 role A] RECV <- peer tells me to go to position (4)
[proc 4 role A] moving from 2 to 4
[proc 4 role A] SEND -> looking up my current pos 4 -> telling peer to go to position (7)
[proc 5 role B] RECV <- peer tells me to go to position (7)
[proc 5 role B] SEND -> looking up my current pos 3 -> telling peer to go to position (6)
[p[proc 5 rroleo c 4 role A] REB] CmovV ing from< - 3p to 7
eer tells me to go to position (6)
[proc 4 role A] moving from 4 to 6
[proc 4 role A] SEND -> looking up my current pos 6 -> telling peer to go to END_MARKER (65535)
[proc 5 role B] RECV <- peer tells me to go to END_MARKER (65535)
[proc 5 role B] SEND -> looking up my current pos 7 -> telling peer to go to END_MARKER (65535)
[proc 5 role B] I have reached the end!
[proc 5 role B] b[oprtoch pro4c reossle eA]s RdonECV e <--> exitipeng
e[r teplrloc me t5o go to END_MARK rER o(1e 6B] cl5535)e
a[pnrupoc comp4 lerted
ole A] I have reached the end!
[proc 4 role A] both processes done -> exiting
[proc 4 role A] cleanup completed
Master: Both processes completed!
Master: Process A exit status: 0
Master: Process B exit status: 0
Master: Cleaning up resources...
Master: Cleanup completed. Game over!
$

```

- Two mailboxes are set up for bidirectional communication.
- Process A and Process B are launched, each attached to shared memory and mailboxes.
- Process A starts at **0**, looks up shared memory, and tells Process B to move to **1**.
- Process B receives the message, moves to **1**, and then looks up its own position to tell Process A to move to **2**.
- Process A receives the message and updates its position to **2**.
- This cycle continues, showing synchronized progression through the maze.
- Both processes detect the **END_MARKER** and announce they've reached the end.
- Processes exit gracefully after cleanup.
- The master program waits for both to finish, reports successful exit statuses, frees resources, and ends the game.